PDF Q&A with Real Page Citations

Technical Documentation & Architecture Guide

Version 0.1.0 | August 2026

1. Overview

DocuMindRAG is a Retrieval-Augmented Generation (RAG) application that allows users to upload PDF documents and ask natural language questions, receiving accurate answers with exact page citations.

> **Key Value Proposition:** Every answer is grounded in the source document with page-level citations, eliminating hallucination and enabling verification.

Core Capabilities

- PDF upload and processing with page-aware chunking
- Hybrid retrieval (dense + sparse vectors) for maximum recall
- Cross-encoder reranking for precision
- LLM-powered answer generation with citation enforcement
- Real-time query interface with source attribution

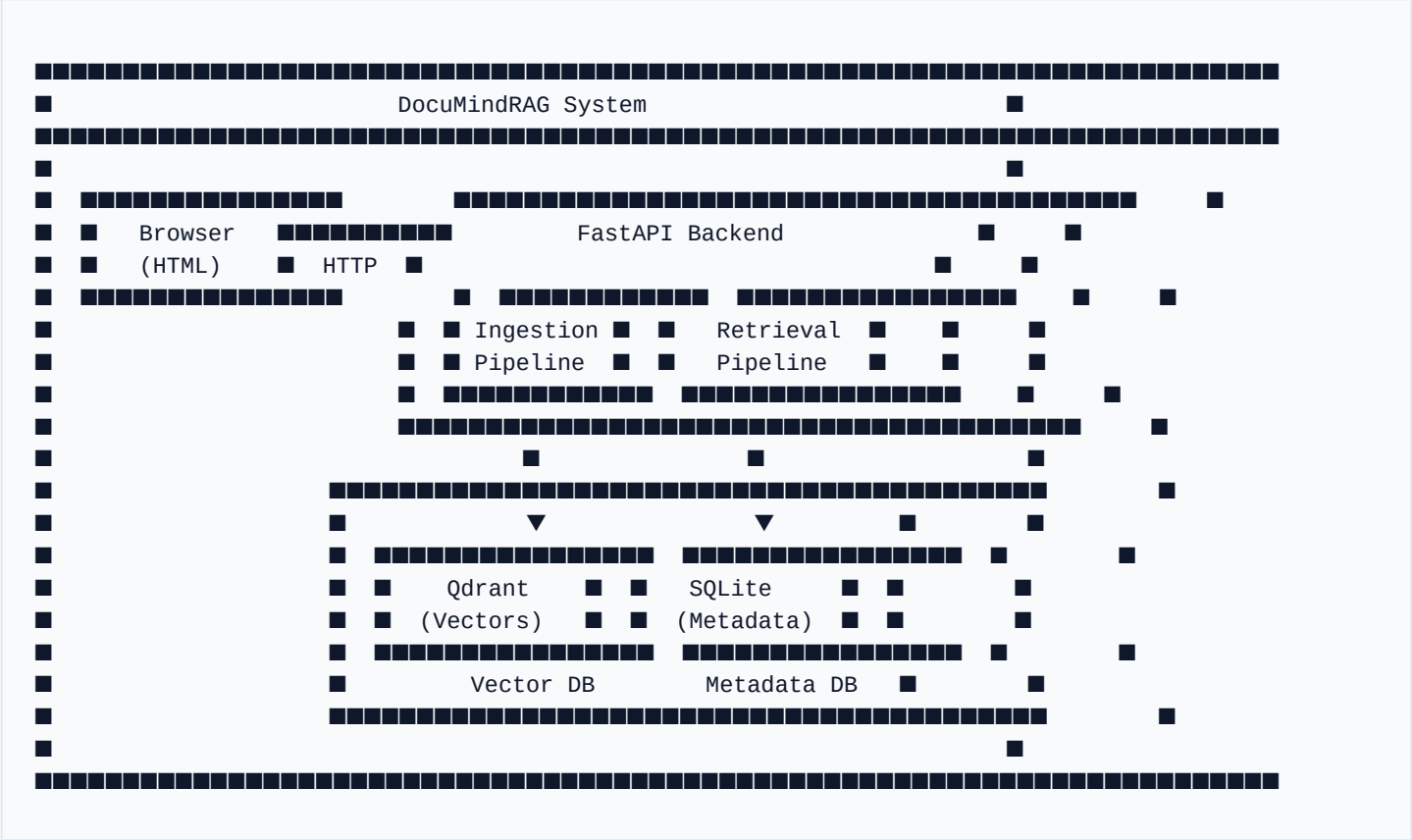
Tech Stack

Component	Technology
Backend	FastAPI (Python 3.13)
Vector Database	Qdrant (Docker)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
Reranker	cross-encoder (ms-marco-MiniLM-L-6-v2)
LLM	Ollama (local) / Gemini (cloud fallback)
PDF Parsing	PyMuPDF
Metadata DB	SQLite (aiosqlite)
Frontend	HTML/CSS/JavaScript

2. RAG Architecture

DocuMindRAG implements a full RAG pipeline with two distinct phases: **Ingestion** (indexing) and **Retrieval** (querying).

System Architecture



2.1 Ingestion Pipeline

The ingestion pipeline transforms raw PDFs into searchable vector embeddings:

1. **PDF Parsing** (PyMuPDF): Extract text from each page
2. **Page-Aware Chunking**: Split pages into 512-token chunks with 128-token overlap
3. **Embedding Generation**: Convert chunks to 384-dim dense vectors using all-MiniLM-L6-v2
4. **Sparse Vector Generation**: Build BM25-style sparse vectors for keyword matching
5. **Qdrant Upsert**: Store dense + sparse vectors with payload metadata
6. **SQLite Storage**: Store document and chunk metadata for citation retrieval

3. Retrieval Pipeline

The retrieval pipeline finds the most relevant chunks and generates grounded answers:

3.1 Hybrid Search

```
Query → [Dense Search] + [Sparse Search] → RRF Fusion → Rerank → Top-K Results
        (Semantic)      (Keywords)         (Combine)     (Precision)   (Final)
```

Step	Method	Purpose	Top-K
1	Dense Search	Semantic similarity via embeddings	20
2	Sparse Search	Exact keyword matching (BM25)	20
3	RRF Fusion	Combine ranked lists (k=60)	20
4	Cross-Encoder Rerank	Precision reranking	5

3.2 Reciprocal Rank Fusion (RRF)

RRF combines results from multiple retrieval methods without requiring score normalization:

```
score(d) = Σ 1/(k + rank_i(d))

where:
- k = 60 (standard constant)
- rank_i(d) = rank of document d in result list i
```

3.3 Answer Generation

The final answer is generated by an LLM with strict citation rules:

- Context chunks are formatted with page markers: `{text} [p.X]`
- System prompt enforces citation requirements
- LLM must cite page numbers for every factual claim

- If context is insufficient, LLM says "I don't have enough information"

> **Citation Prompt:** "For every factual claim, cite the page number like [p.X] or [p.X, p.Y]. If the context doesn't contain enough information, say so explicitly."

4. API Reference

The application exposes a RESTful API at `http://localhost:8000`:

Endpoints

Method	Endpoint	Description
GET	`/`	Web frontend
GET	`/health`	Health check (API + Qdrant status)
POST	`/upload`	Upload and ingest a PDF
GET	`/documents`	List all uploaded documents
DEL	`/documents/{id}`	Delete a document
POST	`/query`	Ask a question

Request/Response Examples

Upload PDF

```
POST /upload
Content-Type: multipart/form-data

Response 200:
{
  "doc_id": "fde6ab7a-...",
  "status": "ready",
  "name": "report.pdf",
  "page_count": 3,
  "chunk_count": 1
}
```

Query

```
POST /query
Content-Type: application/json

{
  "query": "What was the revenue in 2025?",
  "top_k": 5
}

Response 200:
{
  "answer": "The revenue in 2025 was $247.3 million [p.1].",
  "citations": [
    {
      "chunk_id": "...",
      "page_number": 1,
      "chunk_text": "TechVision Inc. achieved record revenue...",
      "retrieval_score": 0.873
    }
  ],
  "timing_ms": 2340.5,
  "model_used": "qwen2.5:0.5b"
}
```

5. Data Model

5.1 SQLite Schema

documents

Column	Type	Description
id	TEXT (UUID)	Primary key
name	TEXT	Original filename
page_count	INTEGER	Number of pages
chunk_count	INTEGER	Number of chunks
status	TEXT	processing / ready / error
created_at	TIMESTAMP	Upload time

chunks

Column	Type	Description
id	TEXT (UUID)	Primary key
doc_id	TEXT	Foreign key → documents
page_number	INTEGER	Page number (1-indexed)
chunk_index	INTEGER	Order within page
text	TEXT	Raw chunk text
char_start	INTEGER	Character offset start

char_end	INTEGER	Character offset end

5.2 Qdrant Collection

Collection name: `documind_chunks`

Vector	Type	Dimensions	Distance
dense	Dense	384	Cosine
sparse	Sparse (BM25)	Variable	Dot

Payload: `{doc_id, page_number, chunk_index, text, char_start, char_end}`

5.3 Chunking Strategy

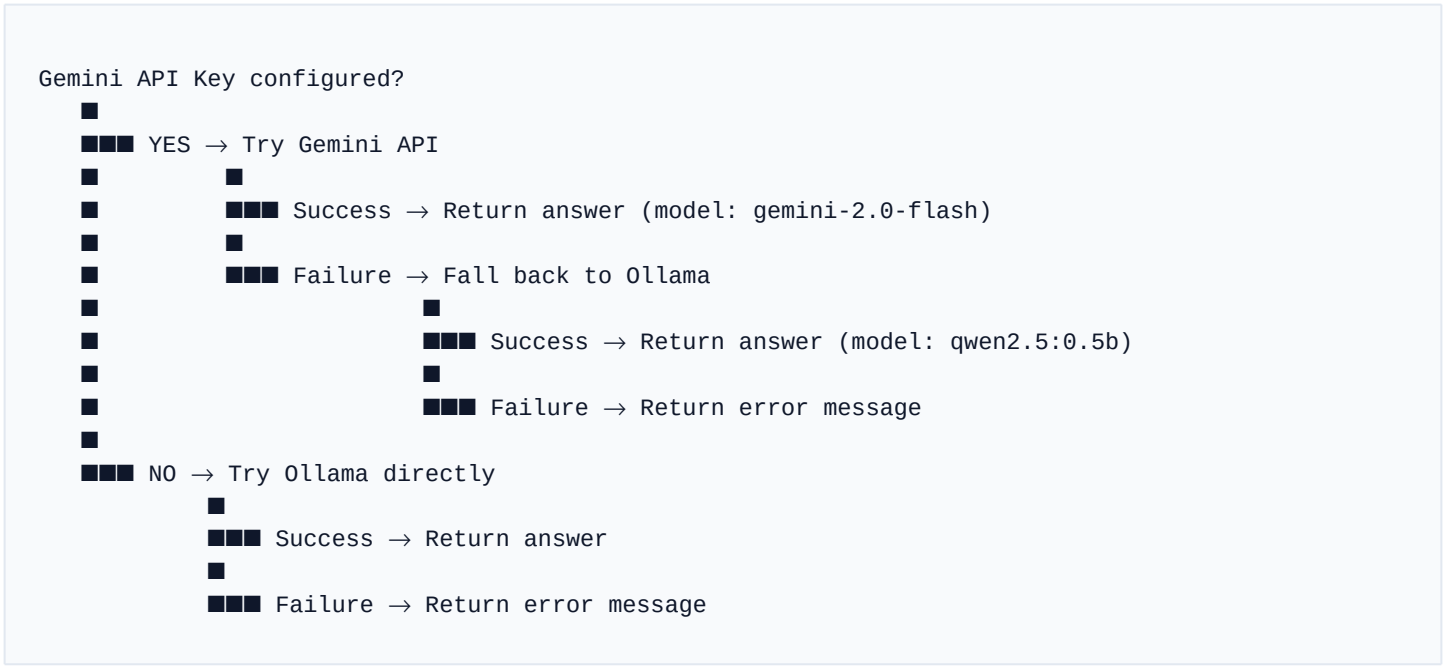
- **Target chunk size:** 512 tokens (~2000 characters)
- **Overlap:** 128 tokens (25%)
- **Max chunk size:** 1024 tokens
- **Merging:** Small pages merge with next page until threshold

6. Configuration

All settings are configured via environment variables or `.env` file:

Variable	Default	Description
`QDRANT_HOST`	localhost	Qdrant server host
`QDRANT_PORT`	6333	Qdrant HTTP port
`EMBEDDING_MODEL`	all-MiniLM-L6-v2	Sentence transformer model
`RERANKER_MODEL`	ms-marco-MiniLM-L-6-v2	Cross-encoder reranker
`GEMINI_API_KEY`	(empty)	Google Gemini API key
`GEMINI_MODEL`	gemini-2.0-flash	Gemini model name
`OLLAMA_MODEL`	qwen2.5:0.5b	Ollama model name
`CHUNK_SIZE_TOKENS`	512	Target chunk size
`CHUNK_OVERLAP_TOKENS`	128	Chunk overlap
`DENSE_TOP_K`	20	Dense retrieval results
`SPARSE_TOP_K`	20	Sparse retrieval results
`RRF_K`	60	RRF constant
`RERANK_TOP_K`	5	Final results after reranking

LLM Fallback Chain


Gemini API Key configured?
YES → Try Gemini API
Success → Return answer (model: gemini-2.0-flash)
Failure → Fall back to Ollama
Success → Return answer (model: qwen2.5:0.5b)
Failure → Return error message
NO → Try Ollama directly
Success → Return answer
Failure → Return error message

7. Deployment & Setup

Prerequisites

- Python 3.11+
- Docker Desktop (for Qdrant)
- Ollama (for local LLM)

Quick Start

```
# Clone and enter the project
cd DocuMindRAG

# Create virtual environment
python -m venv venv
venv\Scripts\activate   # Windows
# source venv/bin/activate  # macOS/Linux

# Install dependencies
pip install -r requirements.txt

# Copy environment config
cp .env.example .env
# Edit .env with your API keys

# Start Qdrant
docker compose up -d

# Pull LLM model (optional, for local inference)
ollama pull qwen2.5:0.5b

# Start the server
python run.py
```

Project Structure

```
DocuMindRAG/
■■■ app/
■   ■■■ main.py              # FastAPI app + routes
■   ■■■ config.py            # Settings (env vars)
■   ■■■ db.py                # SQLite connection + schema
■   ■■■ models.py            # Pydantic schemas
■   ■■■ ingestion/
■   ■   ■■■ pdf_parser.py    # PDF → pages
■   ■   ■■■ chunker.py       # Pages → chunks
■   ■   ■■■ embedder.py      # Chunks → vectors → Qdrant
■   ■■■ retrieval/
■   ■   ■■■ search.py        # Dense + sparse + RRF
```

```
■   ■   ■■■ reranker.py      # Cross-encoder reranking
■   ■   ■■■ answer.py        # LLM answer generation
■   ■■■ services/
■       ■■■ document_service.py  # Orchestration
■■■ static/
■   ■■■ index.html           # Frontend
■   ■■■ css/style.css        # Styling
■   ■■■ js/app.js            # API calls
■■■ sample_pdfs/             # Test documents
■■■ docker-compose.yml       # Qdrant
■■■ requirements.txt
■■■ .env.example
■■■ README.md
```

Access Points

Service	URL
Web Frontend	http://localhost:8000
API Documentation	http://localhost:8000/docs
Qdrant Dashboard	http://localhost:6333/dashboard

8. Design Decisions

Why Hybrid Retrieval?

Dense retrieval captures semantic similarity but may miss exact keyword matches. Sparse retrieval (BM25) excels at keyword matching but lacks semantic understanding. RRF fusion combines both strengths without requiring score normalization.

Why Cross-Encoder Reranking?

Cross-encoders process query-document pairs jointly, providing higher accuracy than bi-encoders used in initial retrieval. The 200-500ms overhead is acceptable for the quality improvement.

Why Page-Aware Chunking?

Standard text splitting breaks page boundaries, making citations inaccurate. Page-aware chunking preserves page information, enabling exact page-level citations in answers.

Why RRF over Linear Combination?

RRF is parameter-free and robust to different score distributions across retrieval methods. Linear combination requires tuning weights and normalizing scores.

9. Future Enhancements

Short-term

- Semantic chunking (sentence-boundary detection)
- Streaming responses for better UX
- Multi-format support (Word, HTML, Markdown)
- Batch ingestion for multiple PDFs

Medium-term

- PostgreSQL migration for production
- User authentication and multi-tenancy
- Custom embedding fine-tuning
- PDF viewer with highlighted citations

Long-term

- Multi-modal RAG (images, tables)

- Agentic RAG with tool use
- Knowledge graph integration
- Enterprise deployment (Kubernetes)